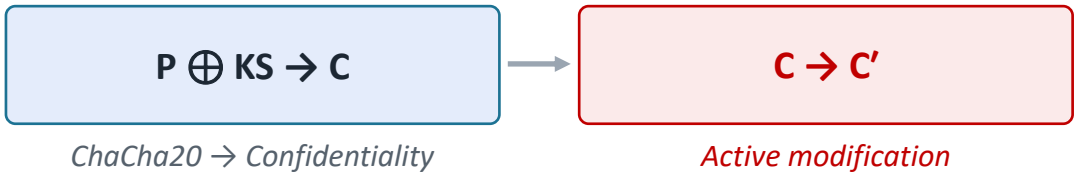
MODULE 2 · LECTURE 8

CS 6530 Applied Cryptography

Authenticated Encryption

Module 2 – Symmetric Cryptography, Hashes and Authentication

Why Not Make One Cipher Do Everything?



How does the receiver know that the ciphertext was modified?

ChaCha20 → Confidentiality + **Poly1305 → Authentication / Integrity component** = **ChaCha20-Poly1305 → AEAD**

AEAD = Authenticated Encryption with Associated Data *(full interface introduced later)*

Compact Operational Model

P —[Key, Nonce / ChaCha20]—→ C then C → C′

Modification detected? ChaCha20 alone: No authentication mechanism.

Think Like an Engineer

Attacker can't learn the plaintext but can modify ciphertext undetected — is confidentiality enough? (No — authentication is also required.)

Insight: *Strong encryption can perfectly protect confidentiality yet give no way to detect ciphertext modification. Poly1305 supplies that authentication component within standardized ChaCha20-Poly1305.*

Secure systems combine cryptographic functions to provide the required security services — using standardized constructions rather than ad-hoc composition.

WHY CONFIDENTIALITY ALONE FAILS

Encryption Does Not Prevent Manipulation

Sender → C → Active Attacker → C′ → Receiver

Scope: unauthenticated XOR-based encryption — stream ciphers / CTR-style encryption

$C = P \oplus KS$
$C' = C \oplus \Delta$
$P' = C' \oplus KS$
Therefore: $P' = P \oplus \Delta$

One-bit illustration
$P_i = 0$
$C_i' = C_i \oplus 1$
$P_i' = 1$

Ciphertext bit change → corresponding plaintext bit change

TRANSFER	ACCOUNT	AMOUNT

$\Delta = P \oplus P^*$; $C' = C \oplus \Delta$; $P' = P^*$ — feasibility depends on knowledge of message format and encoding

Compact Operational Model
Intercept → Modify → Forward / Inject
C → C′ ⇒ P → P′
No key recovery is required.

Think Like an Engineer
Did the attacker break ChaCha20 or recover the key?
(No — the modification simply was not authenticated.)

Insight: *The attacker can defeat the protected communication without defeating the cipher. This is a protocol/system security failure, not necessarily a cipher failure.*

The attacker need not break the cipher to attack the system — malleability of unauthenticated ciphertext can be enough.

What Security Service Is Missing?

Confidentiality
Can unauthorized parties learn the protected data?

Integrity
Has the protected data been modified?

Data-Origin Authentication
Is the protected data authentic under the expected key and protocol context?

Missing: Integrity + Data-Origin Authentication

Protected Data + Authentication Information → VALID or REJECT
An Authentication Tag carries this authentication information (mechanism introduced next).

Compact Operational Model
Protected Data → Auth Check → {VALID | REJECT}
Conceptual only — the keyed MAC mechanism comes in Slide 4.

Think Like an Engineer
Does successful decryption prove the ciphertext was not modified? (No.)

Insight: *Confidentiality asks if an attacker can learn the data; integrity asks if modification can be detected. Authentication additionally relies on secret keying material shared by legitimate participants.*

Secure communication commonly requires confidentiality together with integrity and data-origin authentication.

Combining Encryption and Authentication

$T = MAC(K_m, M)$
$Verify(K_m, M, T) \rightarrow$ VALID or INVALID

$K_e \rightarrow$ Encryption
$K_m \rightarrow$ Authentication

Authentication relies on secret keying material — not merely an unkeyed hash.

1 • Encrypt-and-MAC
$C = Enc(K_e, P)$
$T = MAC(K_m, P)$
Encrypt plaintext and independently authenticate plaintext.

2 • MAC-then-Encrypt
$T = MAC(K_m, P)$
$C = Enc(K_e, P \,||\, T)$
Authenticate first; encrypt plaintext + tag.

3 • Encrypt-then-MAC
$C = Enc(K_e, P)$
$T = MAC(K_m, C)$
Encrypt first; authenticate ciphertext.

Compact Operational Model
Same primitives + different composition ⇒ different security properties
Historical approaches — not equally recommended today.

Think Like an Engineer
If encryption and MAC are each secure, is every way of combining them automatically secure? (No — composition itself is part of the security design.)

How cryptographic primitives are combined matters — not merely which primitives are chosen.

STANDARDIZING THE COMBINATION

From Composition to Authenticated Encryption

Why should every application designer have to combine encryption and authentication correctly?

Authenticated Encryption (AE) = Confidentiality + Integrity / Data-Origin Authentication

Conceptual AE Interface

(C, T) = AE.Enc(K, N, P)

AE.Dec(K, N, C, T) →
P if valid
FAIL if invalid

K — external secret key input to the AE construction
N — nonce
P — plaintext
C — ciphertext
T — authentication tag

Compact Operational Model

(K, N, P) → AE.Enc → (C, T)
(K, N, C, T) → AE.Dec → {P if valid | FAIL if invalid}
One K denotes the AE interface, not raw key reuse.

Think Like an Engineer

What should happen if decryption detects an invalid tag? (REJECT — unauthenticated plaintext must never be accepted or acted upon.)

Insight: *AE changes the application-facing abstraction: instead of separately orchestrating encryption + MAC + composition rules, the application uses one analyzed AE construction.*

Authenticated encryption provides confidentiality and authentication through a single analyzed construction.

BEYOND CONFIDENTIAL PAYLOADS

Some Data Must Remain Visible

VISIBLE DATA	ENCRYPTED PAYLOAD	TAG

May remain visible: protocol metadata • record type/context • sequence or record information • header information

Does visible mean modifiable? Answer: No.

Confidential Payload → Encrypted + Authenticated

Selected Visible Metadata (Used as AAD) → Not Encrypted + Authenticated

AAD = Additional Authenticated Data

Not every protocol header is automatically authenticated as AAD — exact fields are protocol-specific.

Compact Operational Model

Header | Ciphertext | Tag — attacker attempts:
Header → Header'
Modification detected → REJECT

Think Like an Engineer

Must information be secret to require integrity protection? (No.)

Insight: *Confidentiality and integrity are independent — designers choose which fields need both vs. authentication alone. Visible data still needs authentication, answered next via AEAD.*

Protocol metadata may need authenticity even when it does not require confidentiality.

AUTHENTICATED ENCRYPTION WITH ASSOCIATED DATA

AEAD: Protecting Confidential and Visible Data

If visible data is not encrypted, how can the receiver authenticate it?

Conceptual AEAD Interface

(C, T) = AEAD.Enc(K, N, P, A)

AEAD.Dec(K, N, C, A, T) →
P if valid
FAIL if invalid

K — secret key
N — nonce
P — plaintext
A — Additional Authenticated Data (AAD)
C — ciphertext
T — authentication tag

Plaintext (P) →
Encrypted + Authenticated

AAD (A) →
Visible + Authenticated

The nonce is required cryptographic context and must satisfy the AEAD construction's nonce requirements.

Compact Operational Model

P → Encrypt+Authenticate → C
AAD → Authenticate Only | Both → Tag (T)
(K,N) enter the construction; tamper C or A → FAIL.

Think Like an Engineer

Attacker can't decrypt the payload but modifies visible AAD — should the receiver accept? (No — visible means not confidential, not freely modifiable.)

Insight: *AEAD applies confidentiality and integrity differently to different parts of a protocol record, while binding them into one authenticated operation.*

AEAD encrypts the confidential payload while authenticating both the protected payload and selected visible context.

A CONCRETE AEAD CONSTRUCTION

AES-GCM: AEAD from AES

GCM — Galois / Counter Mode

COUNTER
AES Counter Mode → Confidentiality

+

GALOIS
GHASH → Authentication Computation

=

AES-GCM → AEAD

GHASH Hash Subkey

$\mathbf{H = E_k(0^{128})}$

0^{128} = one 128-bit all-zero block

H = hash subkey used by GHASH

$\mathbf{GHASH_h(\ldots)}$ — GHASH parameterized by H

AAD + Ciphertext + Encoded Lengths →[$\mathbf{GHASH_h}$]→ Authentication Value

Encoded lengths = bit-length of AAD and bit-length of ciphertext.
GCM includes these lengths so the AAD/ciphertext boundaries are authenticated.

GHASH uses arithmetic in GF(2^{128}) — the origin of "GALOIS" in AES-GCM

Think Like an Engineer

Why is GCM easier to understand after learning AES-CTR? (Its confidentiality mechanism is counter-mode encryption.)

Insight: *GCM does not replace AES — it specifies how AES counter-mode encryption and Galois-field authentication work together to provide AEAD.*

AES-GCM combines counter-mode confidentiality with Galois-field authentication to provide standardized AEAD.

INSIDE AES-GCM

Why "Galois/Counter Mode"?

COUNTER —

$J_0 = N_{96} \,||\, 0^{31} \,||\, 1$

J_0 = nonce-derived 128-bit initial counter block; it also plays a special role in the tag.

Payload starts at the incremented counter:

$C_1 = P_1 \oplus E_k(N||2)$

$C_2 = P_2 \oplus E_k(N||3)$

GALOIS — Authentication

$H = E_k(0^{128})$

AAD + C + Encoded Lengths $\rightarrow[GHASH_h]\rightarrow$ S

S = GHASH authentication value. Lengths = bit-lengths of AAD and ciphertext (GCM formats/pads this input).

$GF(2^{128})$ — GHASH processing

TAG — Bringing It Together

$J_0 \rightarrow[E_k]\rightarrow E_k(J_0)$

$T = E_k(J_0) \oplus GHASH_h(A,C)$

Conceptual GCM tag relationship.

GHASH input includes standardized padding and encoded lengths.

Engineering Limit — plaintext per invocation (96-bit nonce)

Available blocks: $2^{32}-2 \rightarrow 128(2^{32}-2) = 2^{39}-256$ bits ≈ 64 GiB − 32 bytes

Nonce must not repeat under the same key

Same (K,N) → repeated keystream + GHASH relationships — breaks both confidentiality and authentication.

Compact Operational Model

K branches: $E_k(0^{128})\rightarrow H\rightarrow GHASH_h\rightarrow S$, and $N||2,3,\ldots\rightarrow$AES-CTR$\rightarrow C$

$E_k(J_0) \oplus S \rightarrow$ Tag. Counter=Confidentiality, Galois=Authentication.

Think Like an Engineer

A 96-bit nonce leaves a 32-bit counter. Why isn't max plaintext simply 128×2^{32} bits? (2 counter values are reserved; only $2^{32}-2$ blocks are available for payload.)

AES-GCM combines AES-CTR confidentiality and GHASH-based authentication; its security depends critically on unique nonces and correct tag verification.

AES-GCM • NONCE DISCIPLINE

Nonce Uniqueness Is Non-Negotiable

What happens if the same nonce is reused with the same AES-GCM key?

Same K, same N → same counter sequence → same CTR keystream (KS)
$C_1 = P_1 \oplus KS$ $C_2 = P_2 \oplus KS$

$$C_1 \oplus C_2 = P_1 \oplus P_2$$

CONFIDENTIALITY FAILURE

Is nonce reuse in GCM only a confidentiality problem? **NO**

Authentication consequence: $T = E_k(J_0) \oplus GHASH_h(A,C)$
Nonce reuse → J_0 repeats → $E_k(J_0)$ repeats. $H = E_k(0^{128})$ is already fixed for the key.
Nonce reuse can compromise confidentiality AND authentication.

$N_i \neq N_j$ for distinct invocations under the same key — never reuse a nonce with the same AES-GCM key.
Nonce need not be secret — it must satisfy the uniqueness requirement. (96-bit nonce is the standard/common choice.)

Compact Operational Model

SAFE: $(K,N_1,P_1)\rightarrow(C_1,T_1)$ $(K,N_2,P_2)\rightarrow(C_2,T_2)$ $N_1 \neq N_2$

DANGER: (K,N,P_1) & (K,N,P_2) — same key + nonce → Confidentiality + Authentication at risk

Think Like an Engineer

The nonce is transmitted openly with the ciphertext — is that a security problem? (No — it must be unique, not secret.)

AES-GCM requires nonce uniqueness under each key; reuse can compromise both confidentiality and authentication.

ANOTHER STANDARDIZED AEAD

ChaCha20-Poly1305: From Stream Cipher to AEAD

ChaCha20 + Poly1305 → AEAD

ChaCha20 → Confidentiality

Poly1305 → Authentication Component
(one-time authenticator / MAC)

How is the Poly1305 key obtained?

Counter = 0 → Authentication keying
ChaCha20(K, N, ctr=0) produces a 64-byte block; its first 32 bytes → Poly1305 one-time key.

Counter = 1, 2, 3, … → Payload encryption
Payload encryption begins at counter = 1 and continues — counter 0 is reserved for keying.

AAD + Ciphertext + Encoded Lengths → Poly1305 → Tag
Standardized formatting/padding is applied; encoded lengths bind the AAD and ciphertext lengths into the authenticated input.

Compact Operational Model
(K,N)→ChaCha20: ctr0→key32B→Poly1305 key; ctr1,2,…→KS→C
AAD+C+Lengths→Poly1305(key)→Tag. 256-bit key + 96-bit nonce; must not repeat.

Think Like an Engineer

Why does payload encryption begin at counter 1 rather than counter 0? (Counter 0 derives the one-time Poly1305 key.)

ChaCha20-Poly1305 uses ChaCha20 for confidentiality and to derive Poly1305 keying material, while Poly1305 authenticates the ciphertext and AAD.

AES-GCM vs ChaCha20-Poly1305

Property	AES-GCM	ChaCha20-Poly1305
Encryption	AES-CTR	ChaCha20
Authentication	GHASH	Poly1305
Main operations	AES + GF(2^{128})	ARX + modular authenticator
Common nonce	96 bits	96 bits (IETF)
Nonce reuse	Dangerous	Dangerous
Hardware	Excellent with AES acceleration	No AES-specific hardware required
Software	Strong with suitable AES support	Excellent across many platforms
Output	Ciphertext + Tag	Ciphertext + Tag

(K, N, P, AAD) → (C, T)

Different internal designs — same AEAD abstraction

Both require: Nonce management • Tag verification • Key management • Standardized implementation



Think Like an Engineer

A processor has efficient AES hardware acceleration — could that influence the AEAD choice?

Insight: *Platform and implementation characteristics are legitimate engineering considerations when selecting a permitted AEAD construction.*

AES-GCM and ChaCha20-Poly1305 provide the same AEAD security services through different internal designs and performance trade-offs.

AEAD on the Wire: From Construction to Packet

IPsec ESP with AES-GCM



AAD — Visible + Authenticated

Explicit IV — contributes to protocol-defined GCM nonce
Ciphertext — Encrypted + Authenticated
GCM Tag — Authentication Result

(K, N, P, AAD) →[AES-GCM]→ (C, T)

The protocol defines the packet layout, nonce construction, and AAD fields.

Real Wireshark Capture — IKE Proposal

```
Proposal transforms: 3
Payload: Transform (3)
   Next payload: Transform (3)
   Reserved: 00
   Payload length: 12
   Transform Type: Encryption Algorithm (ENCR) (1)
   Reserved: 00
   Transform ID (ENCR): AES-GCM with a 16 octet ICV (20)
 ⌄ Transform Attribute (t=14,l=2): Key Length: 256
      1... .... .... .... = Format: Type/Value (TV)
      Type: Key Length (14)
      Value: 0100
      Key Length: 256
Payload: Transform (3)
   Next payload: Transform (3)
   Reserved: 00
   Payload length: 8
   Transform Type: Pseudo-random Function (PRF) (2)
   Reserved: 00
   Transform ID (PRF): PRF_HMAC_SHA2_256 (5)
Payload: Transform (3)
   Next payload: NONE / No Next Payload (0)
   Reserved: 00
   Payload length: 8
   Transform Type: Diffie-Hellman Group (D-H) (4)
   Reserved: 00
   Transform ID (D-H): 256-bit random ECP group (19)
```

AES-GCM-256 → AEAD protection; 16-octet ICV = 128-bit tag
HMAC-SHA2-256 → PRF / key-derivation role (not the GCM tag)
ECDH Group 19 → shared-secret establishment

Real IKE negotiation: different mechanisms perform different protocol functions.

TLS Records • IPsec • Secure Messaging • Cloud/Service Traffic
Each protocol defines how its AEAD inputs are constructed.

Think Like an Engineer

Why can some protocol fields remain visible while still requiring authentication?

Insight: *AEAD provides the protection mechanism; the surrounding protocol defines its nonce, AAD, and protected payload.*

Real protocols combine cryptographic mechanisms for distinct jobs and use AEAD to bind visible authenticated context to encrypted data.

AEAD • FAILURE HANDLING

Authentication Failure Means Reject

(C, AAD, Tag) → Authenticated Decryption

TAG VALID
↓
Release Plaintext

TAG INVALID
↓
REJECT

AEAD.Dec(K, N, C, AAD, T) → P or FAIL

Unauthenticated plaintext must not be released, trusted, or acted upon.

C → C′ or AAD → AAD′ ⇒ Tag Verification Fails ⇒ REJECT

Compact Operational Model

Ciphertext + AAD + Tag → Verify/Decrypt → VALID?

YES → Release Plaintext. NO → Reject.

Think Like an Engineer

Plaintext bytes are available, but tag verification fails. Can the application use them? (No.)

Insight: *Authentication failure is a security decision — not merely an error condition.*

Plaintext must be accepted and used only after AEAD authentication succeeds

Five Rules to Take Away

1 — Encryption Alone Is Insufficient

Confidentiality ≠ Integrity/Authentication

2 — Use Standardized AEAD

AES-GCM | ChaCha20-Poly1305

Do not invent ad-hoc compositions.

3 — Manage Nonces Correctly

Never reuse a nonce under the same key

Nonce need not be secret.

4 — Authenticate Required Visible Context

AAD = Visible + Authenticated

5 — Reject Authentication Failure

AEAD.Dec → P or FAIL

Only authenticated plaintext is acceptable.

(K, N, P, AAD) →[AEAD]→ (C, T) • **(K, N, C, AAD, T) →[Verify+Decrypt]→ P or FAIL**

AES-GCM • *ChaCha20-Poly1305*

Think Like an Engineer

A system uses AES-GCM, strong keys, and unique nonces but ignores tag-verification failures — is it secure? (No.)

Insight: *Cryptographic security depends on the primitive, its construction, and how the surrounding system uses it.*

References: NIST SP 800-38D • RFC 8439 • Aumasson, Serious Cryptography • Paar & Pelzl, Understanding Cryptography

Use standardized AEAD, manage nonces correctly, authenticate required context, and reject anything that fails verification.